

2018-2019



Conduite de Projets Objet

Java
CTD 2

Anne-Gwenn Bosser

Plan du cours aujourd'hui – Objectifs pédagogiques

Sujets abordés aujourd'hui:

Héritage, Interfaces, résolution d'appels/types dynamiques ou statiques

Plan des activités:

Récapitulation collective de ce qui a été retenu d'hier (~5 minutes)

Résolution des appels. (~15 minutes)

Exercices: devinettes (~15 minutes)

Plus sur les classes et interfaces (~20 minutes).

Exercice: mise en pratique. (~30 minutes)

Après la séance vous devriez être capable de:

Expliquer dans quels cas un constructeur est appelé.

Expliquer ce qu'est la liaison dynamique (dynamic binding). Décrire quand les appels de méthodes sont résolus statiquement ou dynamiquement.

Mettre en oeuvre sous-typage et délégation pour simuler de l'héritage multiple.

Résolution des appels

- Appel par valeur (*Call by value*)
- Résolution des appels de méthodes
- Constructeurs

Appel des paramètres de méthode

```
public static void test(Person p) {  
    p.age = 10;  
}  
public static void test2(int x) {  
    x = 100;  
}  
  
public static void main(String[] args) {  
  
    int x = 20;  
  
    Person p = new Person("Jimmy Jazz",32);  
  
    System.out.println("X : " + x);  
    System.out.println(p);  
  
    test(p);  
    test2(x);  
  
    System.out.println("X : " + x);  
    System.out.println(p);  
}
```

```
X : 20  
name:Jimmy Jazz age:32  
X : 20  
name:Jimmy Jazz age:10
```


Appel des paramètres de méthode

Il peut sembler, en regardant le code, que le paramètre de la première méthode est passé par référence

⚠ **Call by Value:** les arguments des méthodes sont passés par valeur.

#1: Tout en java est passé par valeur.

#2: Les types des paramètres sont toujours soit des primitives, soit des références (jamais des objets).

```
public static void test(Person p) {  
    p.age = 10;  
}  
public static void test2(int x) {  
    x = 100;  
}
```

En fait p est une référence, on ne manipule pas directement les objets en Java

Notion de signature, prototype de fonction:

- Nom de la fonction
- Visibilité (public, private par défaut)
- Modificateur (static, final,....)
- Type de retour
- Paramètres et leur types

Polymorphisme objet:

“Même signature, fourni un comportement différent”
≈ **Redéfinition** de méthode (***Overriding***)

Polymorphisme paramétrique:

“Différente signature, fourni le même type de comportement”

≈ **Surcharge** de méthode (***Overloading***)

Redéfinir une méthode dans une sous-classe:

- Même signature de fonction avec une implémentation différente
- Liaison tardive (dynamique) *Late Binding*: le choix de la méthode à appeler est résolu lors de l'exécution selon le "vrai" type de l'objet appellant.

⚠ Note: On ne peut pas redéfinir une méthode avec une visibilité plus restreinte

```
class MyClass {  
    public void myMethod() {  
        System.out.Print("Parent  
version");  
    }  
}
```

```
class MySubClass extends MyClass{  
    public void myMethod() {  
        System.out.Print("Child version");  
    }  
}
```


Surcharge des méthodes et des constructeurs:

Même nom de fonction, mais le nombre ou les types des arguments sont différents

Liaison statique (**Static Binding**) : C'est le compilateur qui résout quelle méthode est à appeler

```
public void bonjour() {  
    System.out.println("Bonjour");  
}  
public void bonjour(String name) {  
    System.out.println("Bonjour " + name);  
}
```

⚠ Les opérateurs ne peuvent pas être surchargés en Java. Les opérations sur le type String sont définies par le langage.

Résolution des appels de méthodes

Utilisation de polymorphisme paramétrique: des comportements similaires mais sur des types de données différents.

```
class Dot{}

class Line{}

public class MyClass
    void translate(Dot dot) {
        ...
    }
    void translate(Line line) {
        ...
    }
}
```

```
public class Parent {  
    public int example(){  
        return 0;  
    }  
}
```



```
public class Child extends Parent{  
    public int example() {  
        return 1;  
    }  
}
```

```
public static void Test(Parent p){  
    System.out.println("Test avec param Parent" +  
        "| methode redefinie retourne:" + p.example()+  
        "| type dynamique de l'argument:" + p.getClass());  
}  
public static void Test(Child p){  
    System.out.println("Test avec param Child" +  
        "| methode redefinie retourne:" + p.example()+  
        "| type dynamique de l'argument:" + p.getClass());  
}  
public static void main(String[] args) {  
    Parent Arr[] = {new Parent(), new Child()};  
    for(int i = 0 ; i < Arr.length ; i++){  
        Test(Arr[i]);  
    }  
}
```

Quelle est la sortie?

Résolution des appels de méthodes

run:

```
Test avec param Parent| methode redefinie retourne:0| type dynamique de l'argument:class ctd2.Parent  
Test avec param Parent| methode redefinie retourne:1| type dynamique de l'argument:class ctd2.Child  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Pourquoi ?

Pour la surcharge, l'appel est résolu à l'aide du type statique durant la compilation :

ici: le type des éléments du tableau est Parent

le compilateur résout l'appel par **Test (Parent p)**

Pour la redéfinition, l'appel est résolu à l'aide du type dynamique durant l'exécution
(*Late binding*)

Si l'objet a été créé (lors du new) avec un sous-type de Parent

Alors la méthode appelée est celle définie dans la classe sous-type, donc celle de Child.

Masquage des méthodes statiques

Redéfinition Vs. Masquage:

Les méthodes statiques ne peuvent pas être redéfinies, pour bénéficier du polymorphisme d'objet (runtime)

En effet, leur appel est résolu à la compilation en utilisant le typage statique.

Ces méthodes sont donc masquées dans les sous classe, et doivent être appelées explicitement (nom de la classe, ou type cast).

Masquage des méthodes statiques

```
public class Parent{  
    public static void test_static(){  
        System.out.println("Parent static method");  
    }  
}
```

```
public class Child extends Parent{  
    public static void test_static(){  
        System.out.println("Child static method");  
    }  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
        Parent p = new Parent();  
        Child c = new Child();  
  
        Parent.test_static();  
        Child.test_static();  
  
        p.test_static();  
        c.test_static();  
        p=c;  
        p.test_static();  
    }  
}
```

Qu'est-ce qui est écrit sur la console?

Masquage des méthodes statiques

```
public class Parent{  
    public static void test_static(){  
        System.out.println("Parent static method");  
    }  
}
```

```
public class Child extends Parent{  
    public static void test_static(){  
        System.out.println("Child static method");  
    }  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
        Parent p = new Parent();  
        Child c = new Child();  
  
        Parent.test_static();  
        Child.test_static();  
  
        p.test_static();  
        c.test_static();  
        p=c;  
        p.test_static();  
    }  
}
```

run:

```
Parent static method  
Child static method  
Parent static method  
Child static method  
Parent static method
```

BUILD SUCCESSFUL (total time: 0 seconds)

Constructeurs, Appels des constructeurs

Si (et seulement si) aucun constructeur n'est défini explicitement dans une classe, la compilation génèrera un constructeur par défaut sans paramètres.

Un constructeur peut appeler explicitement un autre constructeur

Cet appel doit être la première instruction du constructeur

On peut utiliser les mots clés `this` et `super`

Exemple: `super()` appelle le constructeur de la superclasse sans paramètres

En cas d'appel mutuel, l'erreur sera détectée à la compilation

Si aucun appel explicite n'est fait, implicitement le constructeur de la classe `Object` sera appelé

Exercices - Devinettes

Décrivez et expliquez les comportements des programmes distribués

Plus sur les classes et interfaces

- Différents types de classe
- Héritage et Interfaces

Elles peuvent être:

Concrètes

Abstraites (en utilisant le mot clé
abstract)

Generiques (paramétrables par un
type)

Internes (ou imbriquées)

et dans ce cas elles peuvent même être
anonymes

Classes Internes

```
package myPackage;

public class Person {

    public String      surname; // public to simplify example
    public ContactDetails details;

    public class ContactDetails
    {
        public String      postcode;
        public String      address;
    }

    public void setAddress(String p, String add){
        details = new ContactDetails();
        details.postcode = p;
        details.address = add;
    }

    public static void main(String[] args) {

        Person a = new Person();
        a.surname = "MR BLABLA";
        a.setAddress("TS13BA", "MIDDLESBROUGH");
        System.out.println("Person: " + a.surname
                            + " living at :"
                            + a.details.postcode
                            + " , "
                            + a.details.address);
    }
}
```

On a ici inclu la définition de la classe ContactDetails dans une autre classe.

```
<terminated> Person (1) [Java Application] C:\Program Files (x86)\Java\jre1.6.0_C
Person: MR BLABLA living at :TS13BA , MIDDLESBROUGH
```



Classes Internes

```
package myPackage;

public class Person {

    public String      surname; // public to simplify example
    public ContactDetails details;

    public class ContactDetails
    {
        public String      postcode;
        public String      address;

        public String getPersonName(){ return Person.this.surname;} // not advised but possible
    }

    public void setAddress(String p, String add){
        details = new ContactDetails();
        details.postcode = p;
        details.address = add;
    }

    public static void main(String[] args) {

        Person a = new Person();
        a.surname = "MR BLABLA";
        a.setAddress("TS13BA", "MIDDLESBROUGH");
        System.out.println("Person: " + a.surname
                           + " living at : "
                           + a.details.postcode
                           + " , "
                           + a.details.address);
        System.out.println("Person: " + a.details.getPersonName());
    }
}
```



```
<terminated> Person (1) [Java Application] C:\Program Files (x86)\Java\jre1.6.0_0
Person: MR BLABLA living at :TS13BA , MIDDLESBROUGH
Person: MR BLABLA
```

Classe Imbriquée anonyme

```
button.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent actionEvent)  
    {  
        System.out.println(actionEvent.getActionCommand()) ;  
    }  
});
```

Classe abstraite

```
public abstract class MyClass{  
    public abstract void myMethod();  
}
```

Ne peut pas être instanciée

Peut être héritée

Peut comporter des méthodes abstraites
(mais ce n'est pas une obligation)

Types Generiques

```
public class Pair<T>{  
    public T first;  
    public T second;  
}
```

Methodes Génériques

```
public <T> void inspect(T t) {  
    System.out.println("T: " + t.getClass().getName());  
}
```

Héritage simple

Interfaces multiples

```
public class MyClass extends OtherClass{  
}
```

La classe `Object` :

Toutes les classes appartiennent à une hiérarchie.

La racine est la classe `Object`.

Voir la documentation des APIs Java:

<https://docs.oracle.com/javase/7/docs/api/>

Une classe peut

Implémenter plusieurs interfaces

N'étendre qu'une seule classe (Pas d'héritage multiple)

Défini un type sans définir une implémentation (comme en UML)

“There are a number of situations in software engineering when it is important for disparate groups of programmers to agree to a "contract" that spells out how their software interacts. Each group should be able to write their code without any knowledge of how the other group's code is written. Generally speaking, interfaces are such contracts.”

<https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>

Une interface peut contenir:

- Des *constantes* (`static final`)

- Des signatures de méthodes

- Des types imbriqués

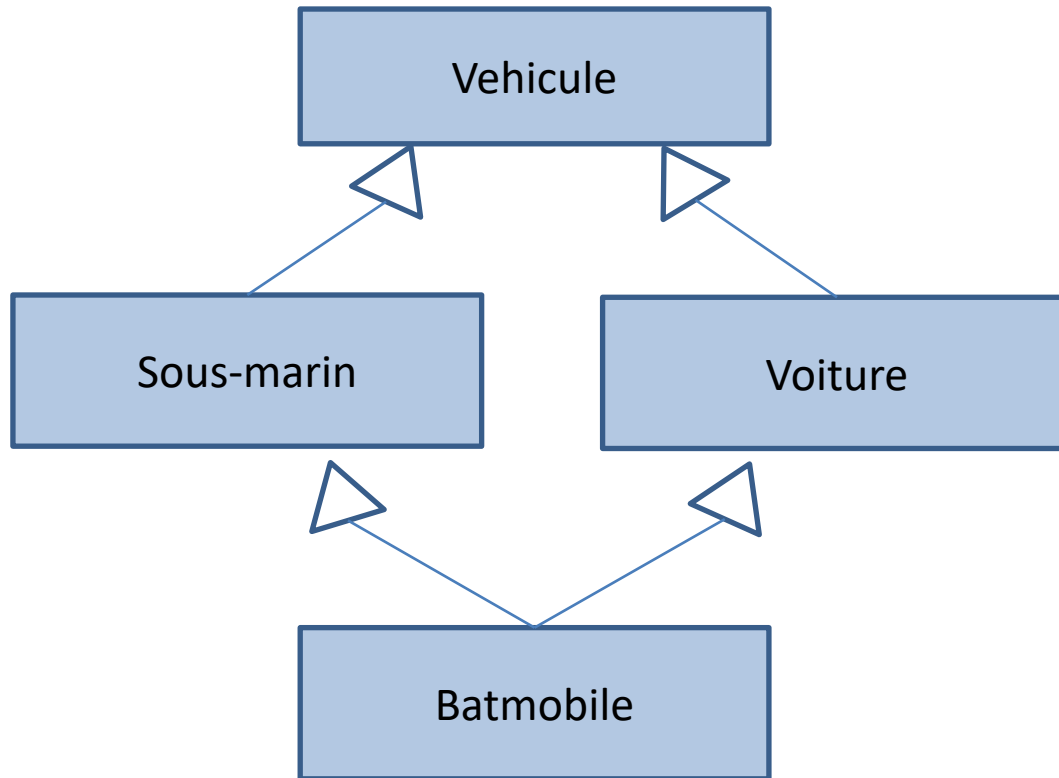
Exemple

```
public interface MyInterface {  
    public void myMethod(int param) ;  
}
```

⚠ la visibilité par défaut est publique, pas *package*.

```
public class MyClass implements Interf1, Interf2{  
    public void myMethod(int param) {  
    }  
}
```

Le “problème” de l’héritage en diamant



Le “problème” de l’héritage en diamant

- Si j’ajoute une méthode *seDeplace* de Véhicule que je redéfini dans les sous-classes Sous-marin et Voiture, comment se déplace la Batmobile?
- L’attribut définissant l’unique propriétaire du véhicule est hérité dans chaque sous-classe, est-il dupliqué ou non dans le Batmobile?
- Et celui définissant la consommation en essence, qui diffère dans sous-marin et Voiture?

Le “problème” de l’héritage en diamant

En Java, on considère ce type de modélisation comme non souhaitable voire mauvais.

Une solution possible, parmi d’autres: considérer non pas la Batmobile comme à la fois une Voiture et un Sous-Marin, mais alternativement Voiture ou Sous-Marin, selon son état.

On définit des classes Voiture et Sous-Marin qui implémentent une interface Véhicule. C’est le code de la classe Batmobile (implémentant elle-même Véhicule) qui décrit quand on doit appeler quelle méthode, par exemple avec un attribut qui conserve son état courant (objet Véhicule de type Voiture ou Sous-Marin), et comment gérer les attributs.

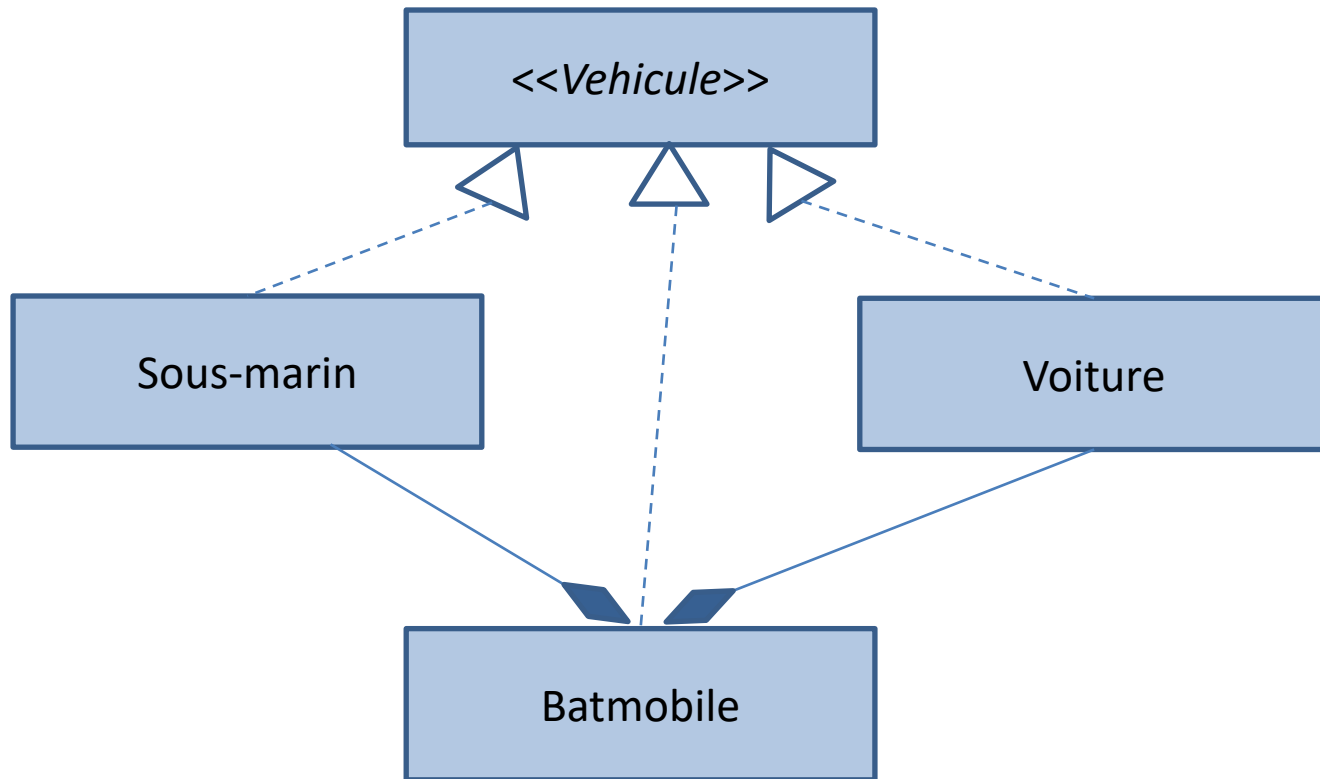
On peut même ranger tous les Véhicule dans un même tableau et appeler en boucle leurs méthodes seDéplace.

Cela correspond bien à ce que vous avez appris:

Héritage = Sous-typage + factorisation de code

Le diagramme UML utilisera sous-typage et aggrégation.

Le “problème” de l’héritage en diamant



Exercice Hulk